Temper

Write Once, Secure Everywhere

A cross-compiled language for security-critical infrastructure at scale

JavaScript | Python | Rust | Java | Lua | C# | and more to come...

THE PROBLEM

SDK maintenance at scale is a security crisis

3,600+
AWS service client libraries

12
language targets

156K+
operation implementations

A single vulnerability in serialization logic must be patched **independently in every language**. The Java SDK v1→v2 migration spanned **7 years**. Three major SDKs hit end-of-support in 2025 — **zero security patches after cutoff**.

.NET SDK v3→v4 has a **3-month maintenance window** vs the 12-month default. The trend is accelerating.

Source: AWS SDKs and Tools maintenance policy, March 2026

THE ANSWER

One source. Six backends. Zero drift.

Temper compiles a single source tree to idiomatic output in 6 languages. Not transpilation — native, typed, importable libraries.

Temper Source → `temper build` → ES Modules | Python 3 | Cargo Crate | Java | Lua | .NET

What Smithy does for AWS

- Generates API clients from models
- Still requires per-language runtime teams
- Security fixes need per-SDK patching
- Multi-year major version migrations

What Temper does better

- Compiles **arbitrary logic**, not just API shapes
- One codebase = one security surface
- Fix once, rebuild, **done everywhere**
- Same-day propagation to all backends

PROOF

A type-safe SQL ORM, running in 6 languages

We built a security-focused ORM with 5 defense layers against SQL injection, compiled it to 6 backends, and built 6 identical todo-list apps to prove it works.

LANGUAGE	FRAMEWORK	ORM IMPORT	SQL INJECTION VULNS
JavaScript	Express + EJS	`from 'orm/src'`	**0**
Python	Flask + Jinja2	`from orm.src import`	**0**
Rust	Axum + askama	`use orm::src::{...}`	**0**
Java	Spring Boot	`import orm.src.*`	**0**
Lua	Raw socket HTTP	`require("orm/src")`	**0**
C#	ASP.NET Razor	`using Orm.Src`	**0**

Full MITRE ATT&CK / CWE analysis conducted per app. Reports published in each app repository.

SECURITY ARCHITECTURE

5 compile-time defense layers

	LAYER	CWE	WHAT IT PREVENTS
1	**SafeIdentifier**	CWE-89	Table/column name injection. Regex-validated, sealed class not exported.
2	**Typed SqlPart**	CWE-89	Value injection. Sealed interface with exhaustive type dispatch.
3	**SqlBuilder**	CWE-89	Structural injection. No `appendRaw`. Structure and data strictly separated.
4	**Changeset**	CWE-915	Mass assignment. `cast()` requires `List<SafeIdentifier>`.
5	**Query Builder**	CWE-400	Unbounded queries. WHERE accepts only `SqlFragment`.

These defenses are **structural, not advisory**. The type system makes it impossible to construct unsafe SQL without deliberately circumventing the API.

FIX ONCE, FIX EVERYWHERE

3 findings. 1 commit. 6 backends patched.

MITRE analysis identified 3 ORM-level concerns. We fixed all 3 in one Temper commit and rebuilt.

FINDING	SEVERITY	ISSUE	FIX	STATUS
ORM-1	Medium	Column names passed as `String`	Routed through `SafeIdentifier`	**Resolved**
ORM-2	Low	`SqlDate` missing quote escaping	Char-by-char escaping	**Resolved**
ORM-3	Low	`SqlFloat64` produces NaN/Infinity	Renders `NULL`	**Resolved**
ORM-4	Info	Escaping-based, not parameterized	Design limitation	Acknowledged

```
// One change in Temper source (changeset.temper.md) colNames.add(pair.key); // raw String
colNames.add(fd.name.sqlValue); // SafeIdentifier → validated // Verified in compiled output across all 6 backends
```

Traditional multi-SDK: **6 PRs, 6 reviews, 6 releases**. Temper: **1 commit, 1 build, done.**

Same prompts. Same goals. Different security.

Two ORMs built with comparable AI prompts (3–4 prompts, same goals). One in Ruby, one in Temper. Both underwent identical MITRE/CWE analysis.

F

SomeORM (Ruby)

CVSS 9.8

VS

A

Generic ORM (Temper)

0 injection vulnerabilities

METRIC	SOMEORM (RUBY)	GENERIC ORM (TEMPER)
SQL injection vulnerabilities	**9 critical/high**	**0**
Identifier validation	None — direct interpolation	SafeIdentifier sealed type
Value escaping	Single-quote only	Typed SqlPart hierarchy (6 types)
Mass assignment protection	None	Changeset `cast()` whitelist
Languages supported	1	**6**
Fix propagation	Manual	**Automatic — 1 commit**

CONVENTIONAL ORM: 9 ATTACK VECTORS

What AI-generated code looks like without Temper

#	CWE	ATTACK SURFACE	CVSS	EXPLOIT
1	CWE-89	Table name injection	9.8	`from("users; DROP TABLE users; --")`
2	CWE-89	SELECT field injection	9.1	`select("(SELECT password FROM admin)")`
3	CWE-89	WHERE raw SQL	9.8	`where("id = #{user_input}")`
4	CWE-89	Comparison field injection	8.6	`eq("id) OR 1=1; --", 1)`
5	CWE-89	GROUP BY injection	8.1	`group_by("user_id; DROP TABLE")`
6	CWE-89	ORDER BY injection	8.1	`order_by("id; DROP TABLE")`
7	CWE-89	Direction injection	5.3	`order_by("id", direction: "ASC; --")`
8	CWE-20	LIMIT type confusion	5.3	`limit("10; DROP TABLE")`
9	CWE-116	Incomplete escaping	6.5	Backslash, NULL byte, encoding

Every one of these is **structurally impossible** in the Temper ORM. Prevented by the type system, not developer discipline.

Temper makes AI-generated code secure by default

The question is not whether AI writes your code. It is whether the language constrains AI to produce safe output.

Without Temper

- AI generates plausible but insecure SQL builders
- String concatenation is the path of least resistance
- Escaping bugs are subtle and language-specific
- Each language needs independent security review
- 9 critical vulns from 3–4 prompts (SomeORM)

With Temper

- Type system prevents unsafe SQL construction
- SafeIdentifier + sealed interfaces = no bypass
- AI cannot accidentally skip validation
- One security review covers all 6 backends
- 0 vulnerabilities from same-effort prompts

> *The best security does not depend on the developer — or the AI — remembering to do the right thing. Temper makes the wrong thing impossible to express.*

CASE STUDY

This entire project was built by AI

Everything you have seen in this deck was produced by a single developer working with Claude. The ORM, the apps, the security analysis, the fixes, and this deck itself.

WHAT	HOW	RESULT
ORM source (Temper)	AI-generated with iterative prompts	5 defense layers, 56 tests, 0 vulns
6 todo-list applications	AI-generated per language	6 frameworks, identical functionality
CI/CD pipeline	AI-generated GitHub Actions	12 repos, automated cascade
MITRE/CWE security audit	AI-conducted analysis	4 ORM findings, 0 app-level injection
Bug fixes (ORM-1/2/3)	AI-identified and AI-fixed	1 commit, 6 backends patched
Verification	AI grepped compiled output	Fixes confirmed in all backends

The AI did not need to know 6 languages to secure 6 backends. It worked in **one language** and the compiler did the rest. This is the workflow AI-driven development needs.

Why AI agents need typed, cross-compiled foundations

As AI coding agents move from autocomplete to autonomous engineering, the infrastructure they build on determines whether their output is safe at scale.

The problem with AI + conventional languages

- AI generates idiomatic code — including idiomatic mistakes
- Ruby, Python, JS all allow raw string SQL with zero friction
- An agent building SDKs must learn security patterns per language
- Review burden multiplies: N languages × M agents × K services
- A single missed escaping call in one language = a CVE

The advantage of AI + Temper

- AI writes to a single type system with enforced invariants
- Sealed interfaces make entire vulnerability classes unrepresentable
- Agents work at the abstraction level, not the language level
- Review burden is constant: 1 source, regardless of target count
- Adding a 7th backend target requires zero security re-audit

Temper does not make AI smarter. It makes AI's mistakes **structurally harmless**.

Review once, ship to every platform

The bottleneck in AI-driven development is not code generation. It is code review. Temper compresses the review surface.

12× → **1×**

review burden today (one per language) → review burden with Temper (one source of truth)

- AI generates a Temper library — human reviews **one codebase**
- Approved code compiles to all targets with **identical security properties**
- No per-language security specialists needed for generated output
- Audit trail is one git history, not twelve

When your organization ships SDKs in 12 languages, the difference between reviewing 12 AI-generated codebases and reviewing 1 is the difference between **possible and impossible**.

AT SCALE

What if SDK security was a single source of truth?

3,600 → **300**

libraries to patch today — Temper sources to maintain

AWS manages 300+ services across 12 languages. A single vulnerability requires coordinated patches across:

- 12 SDK repositories with independent release cycles
- 8+ package registries (npm, PyPI, Maven, NuGet, crates.io, RubyGems, Packagist, Go proxy)
- Daily releases that must be synchronized

With Temper, security logic lives in **one place**. A fix compiles to all targets in a single build. No coordination. No drift.

BEYOND THIS DEMO

Where Temper changes the game

SDK Libraries

API clients, serialization, auth flows, retry logic — write once, ship to every language your customers use.

Security Primitives

Crypto wrappers, input validators, token parsers — the code that must be correct everywhere gets one audit, not twelve.

Data Validation

Schema validators, sanitizers, format parsers — consistent behavior across frontend, backend, and systems languages.

AI-Safe Foundations

Typed guardrails that prevent entire categories of bugs regardless of what generates the code — human or AI.

The numbers

0
SQL injection vulnerabilities across 6 backends

1
commit to fix 3 findings in all 6 languages

56
tests passing after remediation

9
critical vulns in conventional AI-generated ORM

6
language targets from single source

5
compile-time defense layers

Temper does not just reduce development surface area.

It makes security a compile-time guarantee, not a runtime hope.

github.com/temperlang/temper • github.com/notactuallytreyanastasio/generic_orm